

Basic Shell Scripting Reference Guide

Jibon Naher¹ and Samnun Azfar²

¹Lecturer, CSE

²Junior Lecturer, CSE

February 26, 2026

1 Shell?

1.1 What Is a Shell?

A *shell* is a user interface that provides a way to interact with the operating system. It acts as a bridge between the user and the system resources managed by the kernel. The shell itself is an executable program (commonly written in languages such as C) that can interpret user commands, execute programs, and run scripts.

A shell typically performs the following tasks:

- Reads user input (commands).
- Parses and interprets the commands.
- Invokes the appropriate executables or built-in functions.
- Delegates program execution requests to the kernel.
- Displays output or error messages back to the user.

User Input/Script → Shell → Kernel → System Calls / Hardware Resources.

There are two major categories of shells:

- **Command-Line Shells:** Provide a text-based interface. Examples include:
 - `sh` — The original Bourne shell.
 - `bash` — Bourne Again SHell; popular on Linux.
 - `zsh` — Extended Bourne shell with more features.
 - `fish` — User-friendly, modern interactive shell.
 - `csh` / `tcsh` — C-style syntax shells.
- **Graphical Shells:** Provide a GUI for interacting with the system. Examples include:
 - GNOME Shell
 - KDE Plasma Shell
 - Windows Explorer (conceptually a GUI shell)

Regardless of type, all shells serve the same fundamental purpose: enabling the user to control and interact with the system through the operating system.

1.2 How Do We Reach a Shell Prompt?

At a hardware level, a computer system consists of the motherboard, CPU, RAM, and I/O devices. The operating system orchestrates the interactions between these components. In this lab, we aim to understand how the OS, after initializing the hardware and its own internal subsystems, eventually exposes a *shell* to the user. Earlier labs explored executing commands; this section formalizes what a shell is and how the system reaches it.

Let us assume we are booting into a Linux system. The process typically goes as follows:

- The *firmware* (stored on the motherboard—traditionally BIOS, now often UEFI) searches for a *bootloader*. The bootloader usually resides on a disk's EFI System Partition (for UEFI) or in the Master Boot Record (for legacy BIOS). The firmware knows where to look based on its configuration.
- Once the bootloader is loaded into RAM, it begins execution. Bootloaders (such as GRUB) are small programs, often with some components written in assembly. They are commonly structured in multiple stages, but we do not go into those details here.
- The bootloader's primary responsibility is to load the operating system kernel (an executable binary, typically written in C with some assembly). It places the kernel into memory and transfers control to it.
- The kernel initializes core subsystems: memory management, process scheduler, interrupt handlers, and essential device drivers (including storage, input, and display subsystems).
- After initialization, the kernel executes the first userspace program, typically `init` or `systemd`.
- The system and service manager (`systemd`), running as PID 1, continues the boot process by starting required system services.
- One of these services is the login process (such as `agetty` on a TTY or a graphical login manager in GUI environments).
- After a successful login, `systemd` or the login process launches the user's default *shell* (e.g., `bash`, `zsh`, or `fish`). The shell then provides the user with an interactive interface to execute commands and control the system.

Note that the steps in subsection 1.2 occur regardless of whether the system ultimately presents a GUI. Modern Linux distributions typically start a graphical interface in addition to the shell, but the underlying boot sequence remains the same.

1.3 BASH (Bourne Again SHell)

BASH stands for **Bourne Again SHell**, which is the default shell for most Linux distributions and macOS. It is an enhanced replacement for the original Bourne Shell (`sh`).

1.3.1 Components of BASH

BASH consists of several key components that work together to provide a complete command-line environment:

1. **Commands:** Built-in commands like `cd`, `echo`, `pwd`, `export`, etc.

2. **Executables:** External programs stored in directories like /bin, /usr/bin, /usr/local/bin
3. **Keywords:** Reserved words like if, then, else, fi, for, while, do, done
4. **Scripts:** Text files containing sequences of commands with .sh extension
5. **Variables:** User-defined and environment variables (\$PATH, \$HOME, \$USER)
6. **Functions:** Reusable code blocks that can be defined within scripts or interactively

The command `type` lets us check the type, if it's a keyword, command or an executable.

```
1 type ls # ls is an executable, returns the description
```

```
1 type cd # cd is a command as it changes the state of the shell
```

```
1 type if # if is a keyword used in branching statements
```

2 Creating and Running a Shell Script

Step 1 Create a script file

Use a text editor (like gedit or nano):

```
1 gedit hello.sh
```

Step 2 Add the script content

```
1 #!/bin/bash
2 # This is a comment
3 echo "Hello, World!"
```

Observe the `#!/bin/bash` line at the start of the script. The first two ascii characters-`#!` tells the shell that the following parts of the first line is the path to the BASH interpreter. The script file is then passed to the BASH interpreter then the rest of the execution is done through the interpreter itself while the shell we are working on waits.

Step 3 Make it executable

```
1 chmod +x hello.sh # +x gives the script permission to execute
```

Step 4 Run the script

```
1 ./hello.sh
```

Output: Hello, World!

3 Variables in Shell

You can store and use values easily.

variables.sh

```
1 #!/bin/bash
2 name="Max"
3 echo "Hello, $name!"
```

- Always use double quotation when performing variable expansion.
- No spaces around = when assigning values.

4 Environment Variables

Environment variables exist at the process level. They are not a feature of the bash shell, but rather a feature of any program

5 Conditional Statements

conditional.sh

```
1 #!/bin/bash
2 num=10
3 if [ $num -gt 5 ]
4 then
5     echo "Number is greater than 5"
6 else
7     echo "Number is less than or equal to 5"
8 fi
```

Operator	Meaning
-eq	equal
-ne	not equal
-gt	greater than
-lt	less than
-ge	greater or equal
-le	less or equal

Table 1: Numeric comparison operators for [...] and [[...]]

- The [[...]] is the replacement of the deprecated test command. [[acts as a command whose last parameter is]] and within the brackets you are able to execute numeric comparisons which are returned as a boolean result utilized in the if statement.
- The ` = ` operator(with spaces around) is used to compare strings, it cannot compare number.
- The operators mentioned in Table 1 are numeric operators only working on numbers.

6 Arrays

Bash supports one-dimensional indexed arrays, which can store multiple values under a single variable name. Arrays can be created explicitly or implicitly by assignment, and elements are accessed using their index.

Declaring Arrays

You can declare arrays with `declare -a` or by simple assignment.

```
1 # Declaring with initial values
2 fruits=("apple" "banana" "mango")
3
4 # Implicit declaration
5 colors[0]="red"
6 colors[1]="blue"
```

Accessing and Expanding Arrays

Array elements are accessed with `$array[index]`. To expand all elements, use `$array[@]` or `$array[*]`.

```
1 echo "${fruits[0]}"          # prints: apple
2
3 echo "${fruits[@]}"          # prints all elements
4 echo "${#fruits[@]}"         # number of elements
```

Appending to Arrays

Use `+=` or assign to the next index.

```
1 fruits+=("orange")          # append element
2
3 fruits[${#fruits[@]}]="grape" # append manually
4
5 echo "${fruits[@]}"
```

Generating Arrays

You can generate arrays using command substitution, brace expansion, or loops.

```
1 # From command output
2 files=( $(ls *.txt) )
3
4 # Numeric sequence using brace expansion
5 nums={1..10}
6
7 # Generated in a loop
8 for i in {1..5}; do
9     squares+=($(i*i))
10 done
11
12 echo "${squares[@]}"        # 1 4 9 16 25
```

7 Loops

7.1 For Loop

```
1 for i in 1 2 3 4 5;
2 do
3     echo "Number: $i"
4 done
```

7.2 While Loop

```
1 count=1
2 while [ $count -le 5 ];
3 do
4     echo "Count = $count"
5     count=$((count + 1))
6 done
```

8 Reading User Input

read_input.sh

```
1 #!/bin/bash
2 echo "Enter your name:"
3 read name
4 echo "Welcome, $name!"
```

9 Command-Line Arguments

You can pass values to scripts from the terminal.

arguments.sh

```
1 #!/bin/bash
2 echo "First argument: $1"
3 echo "Second argument: $2"
4 echo "Total arguments: $#"
```

Run: ./myscript.sh apple banana

Output:

- First argument: apple
- Second argument: banana
- Total arguments: 2

10 Using System Commands

Shell scripts can automate system tasks easily.

system_commands.sh

```
1 #!/bin/bash
2 echo "Today's date is:"
3 date
4 echo "Current directory is:"
5 pwd
6 echo "Files in this directory:"
7 ls -lh
```

11 Exit Status and Comments

- # → used for comments
- \$? → shows exit status of the last command (0 = success)

11.1 Example

```
1 ls /non/existent/path
2 echo $?
```

(This will likely output a number other than 0, indicating an error.)

12 Arithmetic Expressions

Bash provides built-in support for arithmetic evaluation, which allows you to perform integer calculations directly inside the shell. The most common forms are `$(( ... ))` for substitution and `(( ... ))` for arithmetic evaluation.

- `$((expression))` returns the result and can be assigned or printed.
- `((expression))` evaluates the expression and returns an exit status (0 if true, 1 if false), making it useful in conditions.

Examples:

```
1 # Arithmetic substitution
2 x=$((3 + 5))
3 echo $x    # prints 8
4
5 # Increment
6 ((x++))
7
8 # Using arithmetic evaluation in conditions
9 if (( x > 5 )); then
10     echo "x is greater than 5"
11 fi
```